LAPORAN PRAKTIKUM

STRUKTUR DATA

“Laporan Praktikum Pekan 4”

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira

DAPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum **Praktikum Struktur Data** untuk pertemuan pekan ke-4 ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk mendokumentasikan hasil praktikum mengenai struktur data **Queue**. Fokus utama pada pekan ini adalah memahami prinsip *First In First Out* (FIFO) melalui berbagai metode implementasi, mulai dari penggunaan *library* standar Java berbasis LinkedList, implementasi manual menggunakan array sirkular, hingga teknik pembalikan data menggunakan bantuan Stack.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Semoga laporan ini dapat memberikan manfaat bagi pembaca dalam memahami mekanisme antrian dalam pemrograman Java.

Padang, 30 April, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN 3
2.1 Class QueueArray_2511532014 3
2.2 Class QueueArrayDriver_2511532014 5
2.3 Class QueueLlinkedList_2511532014 6
2.4 Class ReverseData_2511532014 7
2.5 Class IterasiQueue_2511532014 8
BAB III KESIMPULAN 10
DAFTAR PUSTAKA 11

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data Queue atau antrian adalah sekumpulan data linear yang mengikuti prinsip FIFO (First In First Out), di mana elemen yang pertama kali dimasukkan akan menjadi yang pertama kali dikeluarkan. Berbeda dengan Stack yang bersifat LIFO, Queue mensimulasikan antrian di dunia nyata, seperti antrian bank atau proses pencetakan dokumen pada printer.

Dalam implementasinya, Queue memiliki dua ujung utama: *Front* (depan) untuk mengambil data dan *Rear* (belakang) untuk menambah data. Praktikum ini mencakup implementasi Queue statis menggunakan array yang dioptimalkan dengan logika modulo untuk mencegah pemborosan memori, serta implementasi dinamis menggunakan *library* Java Queue dan LinkedList.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep, karakteristik, dan mekanisme kerja struktur data Queue (FIFO).
2. Mampu mengimplementasikan operasi dasar Queue seperti enqueue (tambah), dequeue (hapus), dan peek/front.
3. Memahami penggunaan array sirkular untuk mengimplementasikan Queue secara manual guna mengoptimalkan kapasitas memori.
4. Mampu melakukan iterasi pada elemen Queue menggunakan Iterator.
5. Mampu mengintegrasikan Queue dengan Stack untuk melakukan pembalikan urutan data (*reverse*).

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini meliputi:

1. Peningkatan Logika Pemrograman: Melatih cara berpikir sistematis dalam mengelola urutan data berdasarkan prioritas tumpukan.
2. Pemahaman Alur Data: Mahasiswa memahami alur pemrosesan data yang adil (berdasarkan urutan kedatangan).

3. Optimasi Algoritma: Melatih penggunaan logika sirkular (modulo) dalam array untuk efisiensi indeks.
4. Keterampilan Integrasi: Mahasiswa mampu menggabungkan berbagai struktur data (Queue dan Stack) untuk memecahkan masalah logika pemrograman.
5. Penguasaan Java Collections: Meningkatkan kemampuan dalam menggunakan LinkedList dan Iterator yang merupakan bagian penting dari Java Collections Framework.

BAB II
PEMBAHASAN

2.1 Class QueueArray_2511532014

2.1.1 Langkah Pengerjaan

1. Mendefinisikan atribut front_2014, rear_2014, size_2014, dan capacity_2014.
2. Menggunakan array sebagai penampung data statis dengan ukuran tertentu.
3. Mengimplementasikan metode isFull dan isEmpty berdasarkan nilai size_2014.
4. Membuat metode enqueue_2511532014 dengan logika sirkular (rear + 1) % capacity untuk menambah elemen di belakang.
5. Membuat metode dequeue_2014 dengan logika sirkular (front + 1) % capacity untuk mengambil elemen di depan.
6. Menyusun metode display_2014 untuk menampilkan isi antrian yang tersedia.

2.1.2 Kode Program

```java
package Pekan4_2511532014;

public class QueueArray_2511532014 {
    int front_2014, rear_2014, size_2014;
    int capacity_2014;
    int array_2014[];

    public QueueArray_2511532014(int capacity_2014) {
            this.capacity_2014 = capacity_2014;
            front_2014 = this.size_2014 = 0;
            rear_2014 = capacity_2014 - 1;
            array_2014 = new int[this.capacity_2014];
    }

    boolean isFull_2511532014(QueueArray_2511532014 queue_2014) {
            return (queue_2014.size_2014 == queue_2014.capacity_2014);
    }

    boolean isEmpty_2511532014(QueueArray_2511532014 queue_2014) {
            return (queue_2014.size_2014 == 0);
    }

    void enqueue_2511532014 (int item_2014) {
            if (isFull_2511532014(this))
                    return;
            this.rear_2014 = (this.rear_2014 + 1) % this.capacity_2014;
            this.array_2014[this.rear_2014] = item_2014;
            this.size_2014 = this.size_2014 + 1;
            System.out.println(item_2014 + " enqueue to queue");
    }

```

```java
    int dequeue_2014() {
            if (isEmpty_2511532014 (this))
                    return Integer.MIN_VALUE;
            int item_2014 = this.array_2014[this.front_2014];
            this.front_2014 = (this.front_2014 + 1) % this.capacity_2014;
            this.size_2014 = this.size_2014 - 1;
            return item_2014;
    }

    int front_2014() {
            if (isEmpty_2511532014(this))
                    return Integer.MIN_VALUE;

            return this.array_2014[this.front_2014];
    }

    int rear_2014() {
            if (isEmpty_2511532014(this))
                    return Integer.MIN_VALUE;
            return this.array_2014[this.rear_2014];
    }

    void display_2014() {
            int i;
            if (front_2014 == rear_2014) {
                    System.out.printf("\nAntrian Kosong\n");
                    return;
            }
            //kunjungi dari belakang dan cetak
            for (i = front_2014; i < rear_2014; i++) {
                    System.out.printf(" %d <-- ", array_2014[i]);
            }
            return;
    }

}
```

2.1.3 Analisis

Implementasi Queue menggunakan array manual ini menerapkan teknik Circular Queue. Penggunaan operator modulo (%) sangat krusial; hal ini memungkinkan indeks rear dan front untuk kembali ke awal array (indeks 0) setelah mencapai kapasitas maksimum, asalkan ada slot kosong yang tersedia akibat proses dequeue. Tanpa logika sirkular, array akan cepat dianggap penuh meskipun terdapat ruang kosong di bagian depan akibat penghapusan data. Variabel size_2014

digunakan sebagai indikator utama untuk mengecek kondisi penuh atau kosongnya antrian secara efisien.

2.2 Class QueueArrayDriver_2511532014

2.2.1 Langkah Pengerjaan

1. Instansiasi objek QueueArray_2511532014 dengan kapasitas 1000.
2. Menambahkan empat data (10, 20, 30, 40) ke dalam antrian.
3. Menampilkan elemen terdepan dan terbelakang menggunakan metode front() dan rear().
4. Menjalankan dequeue_2014() untuk menghapus satu data terdepan dan menampilkan kembali status antrian.

2.2.2 Kode Program

```java
package Pekan4_2511532014;

public class QueueArrayDriver_2511532014 {
    public static void main(String[]args) {
            QueueArray_2511532014 queue = new QueueArray_2511532014(1000);
            queue.enqueue_2511532014(10);
            queue.enqueue_2511532014(20);
            queue.enqueue_2511532014(30);
            queue.enqueue_2511532014(40);
            System.out.println("Item di depan "+ queue.front_2014());
            System.out.println("Item paling belakang " + queue.rear_2014());
            System.out.println("Tampilkan queue");
            queue.display_2014();
            System.out.println();
            System.out.println(queue.dequeue_2014()+ " dihapus dari queue");
            System.out.println("Item di depan "+ queue.front_2014());
            System.out.println("Item paling belakang " + queue.rear_2014());
            System.out.println("Tampilan queue setelah satu data dihapus ");
            queue.display_2014();
    }

}


```

2.2.3 Output

```
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
```

```
40 enqueue to queue
Item di depan 10
Item paling belakang 40
Tampilkan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
Item di depan 20
Item paling belakang 40
Tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

2.2.4 Analisis

Driver ini memverifikasi bahwa prinsip FIFO berjalan dengan benar. Saat data 10, 20, 30, dan 40 dimasukkan, 10 berada di posisi front dan 40 di posisi rear. Setelah operasi dequeue, data 10 (yang paling pertama masuk) dihapus, sehingga data 20 naik menjadi elemen terdepan baru. Hal ini membuktikan bahwa pengambilan data pada Queue selalu dilakukan pada ujung depan (*front*)

2.3 Class QueueLlinkedList_2511532014

2.3.1 Langkah Pengerjaan

1. Menggunakan interface Queue dan implementasi LinkedList dari java.util.
2. Menambahkan elemen 0 hingga 5 menggunakan perulangan for dan metode add().
3. Menghapus elemen terdepan dengan remove() dan melihat elemen terdepan dengan peek().

2.3.2 Kode Program

```java
package Pekan4_2511532014;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511532014 {

    public static void main(String[] args) {
            Queue<Integer> q_2511532014 = new LinkedList<>();
            // tambah element
            for (int i = 0; i < 6; i++)
                    q_2511532014.add(i);
            //menampilkan isis antrian
            System.out.println("Element Antrian " + q_2511532014);
            //untuk menghapus kepala antrian
            int hapus_2014 = q_2511532014.remove();
            System.out.println("Hapus Element = "+ hapus_2014);
            System.out.println(q_2511532014);
```

```
            //untuk melihat antrian terdepan
            int depan_2014 = q_2511532014.peek();
            System.out.println("Kepala Antrian = " + depan_2014);

            int banyak_2014 = q_2511532014.size();
            System.out.println("Size Antrian = "+ banyak_2014);
    }

}
```

2.3.3 Output

```
Element Antrian [0, 1, 2, 3, 4, 5]
Hapus Element = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

2.3.4 Analisis

Implementasi ini memanfaatkan keunggulan LinkedList sebagai basis data dinamis. Berbeda dengan array, LinkedList tidak memiliki batasan kapasitas statis dan lebih efisien dalam proses penambahan serta penghapusan elemen karena tidak perlu melakukan pergeseran indeks memori. Penggunaan metode peek() sangat berguna untuk memeriksa data terdepan tanpa mengubah kondisi antrian.

2.4 Class ReverseData_2511532014

2.4.1 Langkah Pengerjaan

1. Menyiapkan sebuah Queue berisi data [1, 2, 3].
2. Memindahkan seluruh elemen Queue ke dalam sebuah Stack menggunakan metode push(remove()).
3. Mengembalikan data dari Stack ke Queue menggunakan metode add(pop()).

2.4.2 Kode Program

```
package Pekan4_2511532014;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532014 {

    public static void main(String[] args) {
            Queue<Integer> q_2511532014 = new LinkedList<Integer>();
```

```
        q_2511532014.add(1);
        q_2511532014.add(2);
        q_2511532014.add(3); //[1, 2, 3]
        System.out.println("Sebelum reverse "+ q_2511532014);
        Stack<Integer> s_2511532014 = new Stack<Integer>();
        while (!q_2511532014.isEmpty()) {
            s_2511532014.push(q_2511532014.remove());
        }
        while (!s_2511532014.isEmpty()) {
            q_2511532014.add(s_2511532014.pop());
        }
        System.out.println("Sesudah reverse: "+ q_2511532014);

    }

}
```

2.4.3 Output

```
Sebelum reverse [1, 2, 3]
Sesudah reverse: [3, 2, 1]
```

2.4.4 Analisis

Proses pembalikan (*reverse*) data Queue secara efektif dilakukan dengan memanfaatkan sifat berlawanan antara Queue (FIFO) dan Stack (LIFO). Saat elemen Queue dimasukkan ke Stack, urutannya menjadi terbalik (elemen terakhir Queue berada di puncak Stack). Ketika dikeluarkan kembali dari Stack, elemen-elemen tersebut masuk ke Queue dalam urutan yang sudah terbalik sepenuhnya. Ini adalah teknik standar untuk membalikkan urutan koleksi data linear.

2.5 Class IterasiQueue_2511532014

2.5.1 Langkah Pengerjaan

1. Membuat Queue bertipe data String.
2. Memasukkan kata-kata pembentuk kalimat ke dalam antrian.
3. Menggunakan objek Iterator untuk menelusuri dan mencetak setiap elemen secara berurutan.

2.5.2 Kode Program

```
package Pekan4_2511532014;

import java.util.Iterator;
import java.util.LinkedList;
```

```java
import java.util.Queue;

public class IterasiQueue_2511532014 {

	public static void main(String[] args) {
		Queue<String> q_2511532014 = new LinkedList<>();

		q_2511532014.add("Praktikum");
		q_2511532014.add("Struktur");
		q_2511532014.add("Data");
		q_2511532014.add("Dan");
		q_2511532014.add("Algoritma");
		Iterator<String> iterator_2511532014 = q_2511532014.iterator();
		while (iterator_2511532014.hasNext()) {
			System.out.print(iterator_2511532014.next()+ " ");
		}

	}

}
```

2.5.3 Output

```
Praktikum Struktur Data Dan Algoritma
```

2.5.4 Analisis

Penggunaan Iterator memungkinkan penelusuran elemen Queue tanpa harus mengeluarkan atau menghapus data tersebut dari antrian. Metode hasNext() memastikan perulangan tetap berjalan selama masih ada elemen berikutnya, sementara next() mengambil data saat ini dan menggeser kursor ke elemen selanjutnya. Hal ini sangat berguna untuk menampilkan laporan atau log data tanpa merusak struktur antrian yang sedang aktif.

BAB III
KESIMPULAN

Berdasarkan praktikum pekan ke-3, dapat disimpulkan bahwa:

1. Queue adalah struktur data FIFO yang sangat efektif untuk manajemen urutan tugas atau data yang memerlukan keadilan proses berdasarkan kedatangan.
2. Implementasi Queue manual menggunakan Array Sirkular (dengan logika modulo) mampu mengatasi kelemahan array statis biasa dalam hal pemborosan ruang memori.
3. Struktur data Stack dan Queue dapat dikombinasikan untuk melakukan manipulasi data yang kompleks, seperti membalikkan urutan elemen (reverse).
4. Penggunaan Java Collections (Queue dan LinkedList) mempermudah pengembangan karena ukurannya yang dinamis dan ketersediaan metode siap pakai.

DAFTAR PUSTAKA

Laporan Tugas

1. Class AntrianLoket_2511532014

a. Langkah Pengerjaan

i. Mendeklarasikan atribut queue_2014[] bertipe String untuk menyimpan nama pelanggan, serta variabel front_2014, rear_2014, dan max_2014.

ii. Menginisialisasi tumpukan melalui konstruktor dengan mengatur nilai front_2014 dan rear_2014 ke -1 untuk menandai antrian kosong.

iii. Membuat fungsi isEmpty_2014() dan isFull_2014() untuk memvalidasi status kapasitas array.

iv. Mengimplementasikan metode enqueue_2014() untuk menambahkan nama pelanggan ke posisi belakang (rear) antrian.

v. Mengimplementasikan metode dequeue_2014() untuk melayani pelanggan di posisi depan (front) dan menggeser penanda indeks.

vi. Membuat metode display_2014() untuk menampilkan daftar seluruh pelanggan yang sedang mengantri.

vii. Mengimplementasikan metode reverse_2014() menggunakan teknik swapping untuk membalikkan urutan nama dalam antrian.

viii. Menyusun program utama (main) dengan menu interaktif menggunakan do-while dan switch-case.

b. Kode Program

```java
package Pekan4_2511532014;

import java.util.Scanner;

public class AntrianLoket_2511532014 {

    String queue_2014[];
    int front_2014, rear_2014, max_2014;

    // Constructor
    public AntrianLoket_2511532014(int max_2014) {
        this.max_2014 = max_2014;
        queue_2014 = new String[max_2014];
        front_2014 = -1;
        rear_2014 = -1;
    }

    // Cek kosong
    public boolean isEmpty_2014() {
        return front_2014 == -1;
    }

    // Cek penuh
    public boolean isFull_2014() {
        return rear_2014 == max_2014 - 1;
    }

    // Enqueue
```

```java
    public void enqueue_2014(String data_2014) {
        if (isFull_2014()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_2014()) {
                front_2014 = 0;
            }
            rear_2014++;
            queue_2014[rear_2014] = data_2014;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    // Dequeue
    public void dequeue_2014() {
        if (isEmpty_2014()) {
            System.out.println("Antrian kosong!");
        } else {
            String data_2014 = queue_2014[front_2014];
            System.out.println(data_2014 + " telah dilayani");

            if (front_2014 == rear_2014) {
                front_2014 = rear_2014 = -1;
            } else {
                front_2014++;
            }
        }
    }

    // Display
    public void display_2014() {
        if (isEmpty_2014()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            int no_2014 = 1;
            for (int i_2014 = front_2014; i_2014 <= rear_2014; i_2014++) {
                System.out.println(no_2014 + ". " + queue_2014[i_2014]);
                no_2014++;
            }
        }
    }

    // Reverse
    public void reverse_2014() {
        if (isEmpty_2014()) {
            System.out.println("Antrian kosong!");
        } else {
            int start_2014 = front_2014;
            int end_2014 = rear_2014;

            while (start_2014 < end_2014) {
                String temp_2014 = queue_2014[start_2014];
                queue_2014[start_2014] = queue_2014[end_2014];
```

```
                queue_2014[end_2014] = temp_2014;

                start_2014++;
                end_2014--;
            }

            System.out.println("Antrian berhasil dibalik!");
        }
    }

    // Main Program
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        AntrianLoket_2511532014 antrian_2014 = new AntrianLoket_2511532014(10);

        int pilihan_2014;
        do {
                System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
                System.out.println("1. Tambah Antrian");
                System.out.println("2. Hapus Antrian");
                System.out.println("3. Tampilkan Antrian");
                System.out.println("4. Reverse");
                System.out.println("5. Keluar");
                System.out.print("Pilih menu: ");
                pilihan_2014 = input.nextInt();
                input.nextLine();

                switch (pilihan_2014) {
                    case 1:
                        System.out.print("Masukkan nama pelanggan: ");
                        String nama_2014 = input.nextLine();
                        antrian_2014.enqueue_2014(nama_2014);
                        break;

                    case 2:
                        antrian_2014.dequeue_2014();
                        break;

                    case 3:
                        antrian_2014.display_2014();
                        break;

                    case 4:
                        antrian_2014.reverse_2014();
                        antrian_2014.display_2014();
                        break;

                    case 5:
                        System.out.println("Program selesai");
                        break;

                    default:
```

```
                        System.out.println("Pilihan tidak valid!");
                    }

                } while (pilihan_2014 != 5);

                input.close();
            }
        }

```

c. Output

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: yanto
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: faiz
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. yanto
2. faiz

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik!
Isi antrian:
1. faiz
2. yanto

```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
faiz telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```

d. Analisis

Program ini mensimulasikan sistem antrian loket di dunia nyata menggunakan array linear. Berbeda dengan QueueArray yang menggunakan logika sirkular, program ini menggunakan pendekatan linear sederhana di mana front_2014 dan rear_2014 terus bergerak maju. Analisis pada metode reverse_2014() menunjukkan penggunaan algoritma **Two-Pointer Technique**, di mana penanda start bergerak maju dan end bergerak mundur untuk menukar nilai elemen. Teknik ini sangat efisien karena hanya memerlukan waktu O(n/2) tanpa membutuhkan struktur data tambahan seperti Stack. Kelemahannya, pada implementasi linear ini, ruang array di bagian depan yang sudah kosong setelah dequeue tidak dapat digunakan kembali secara otomatis kecuali dilakukan reset indeks saat antrian benar-benar kosong.